

Coffee Shop Sales Performance, Revenue Optimization & Consumer Purchase Behavior Analytics

✓ **Objective**

The objective of this project is to perform end-to-end sales analytics using Python to identify revenue trends, customer purchasing behavior, product performance, and store-wise sales patterns.

The project aims to help coffee shop management make data-driven decisions regarding inventory management, product strategy, pricing optimization, and sales growth.

Problem Statement

The coffee shop generates a large number of daily transactions across multiple products, categories, and store locations. However, the management lacks clear insights into sales performance, customer purchasing behavior, peak business hours, and product profitability.

Using Python and Data Analytics techniques, this project aims to:

Import and preprocess coffee shop sales data using Pandas. Identify and handle missing values, duplicate records, and data inconsistencies. Perform Exploratory Data Analysis (EDA) to understand sales trends and customer purchasing patterns. Analyze monthly, daily, and hourly sales performance. Identify top-selling products and product categories. Evaluate store-wise sales performance. Determine the relationship between transaction quantity and revenue generation. Identify peak business hours and customer purchasing trends. Generate visualizations and business insights using Matplotlib and Seaborn. Provide data-driven recommendations to improve sales and business performance.

✓ ***Expected Outcomes***

The analysis of the Coffee Shop Sales dataset is expected to generate valuable insights into sales performance, customer purchasing behavior, product demand, and store operations.

1. Enhanced Understanding of Sales Performance

Identify monthly, daily, and hourly sales trends.

Understand how sales fluctuate across different periods.

2. Improved Data Quality

Detect and handle missing values, duplicate records, and formatting issues.

Create a clean dataset suitable for accurate analysis and decision-making.

3. Product Performance Analysis

Determine the most popular products based on quantity sold.

Identify products contributing the highest revenue.

Understand customer preferences across product categories.

4. Revenue Analysis

Measure revenue generated by different products and categories.

Identify the major contributors to overall sales.

Provide recommendations to maximize profitability.

5. Peak Sales Period Identification

Determine peak business hours.

Identify the busiest days and months.

Help management optimize staffing and inventory planning.

6. Store Performance Evaluation

Compare sales performance across different store locations.

Identify top-performing and underperforming stores.

Support strategic business decisions.

7. Customer Purchasing Behavior Analysis

Understand when customers are most likely to purchase products.

Analyze transaction volume and purchase frequency patterns.

8. Actionable Business Insights

Discover opportunities to increase revenue and improve operational efficiency.

Support data-driven decision-making for business growth.

9. Strategic Recommendations

- Recommend promotional strategies for low-performing products.
- Suggest inventory optimization methods.
- Provide business strategies to improve sales and customer satisfaction.

Domain

Retail Analytics / Food & Beverage Analytics

Dataset Column Description

Column Name	Description
transaction_id	Unique transaction identifier
transaction_date	Date of transaction
transaction_time	Time of transaction
transaction_qty	Quantity of products purchased
store_id	Store identifier
store_location	Store location
product_id	Product identifier
unit_price	Price per unit
product_category	Product category
product_type	Product type
product_detail	Product description
Sales	Revenue generated (Qty × Unit Price)

Business Questions

Sales Analysis

- 1.Which month generated the highest revenue?
- 2.Which day recorded the highest sales?
- 3.What are the peak business hours?
- 4.Which store location performs best?

Product Analysis

- 1.Which products are sold most frequently?
- 2.Which product category generates the highest revenue?

3.Which products contribute the least revenue?

4.Customer Trend Analysis

Are weekends busier than weekdays?

1.What time do customers make the most purchases?

2.Which months generate the highest customer traffic?

Revenue Analysis

1.What is the total revenue generated by the coffee shop?

2.Which store contributes the highest revenue?

3.Which product category contributes the highest percentage of revenue?

Business Improvement Questions

1.Which products should be promoted to increase sales?

2.What strategies can improve sales during low-performing periods?

✓ ***Stage 1 Completion***

- ✓ Problem Definition
- ✓ Expected Outcomes
- ✓ Dataset Source
- ✓ Dataset Description
- ✓ Business Questions Defined

```
1 import pandas as pd
2
3 url = "https://raw.githubusercontent.com/sivaranjania123-sketch/coffee-shop
4
5 df = pd.read_csv(url)
6
7 print(df.shape)
8 df.head()
```

```
(149116, 10)
```

	transaction_id	transaction_date	transaction_qty	store_id	store_location
0	1	01-01-2023	2	5	Lower Manhattan
1	2	01-01-2023	2	5	Lower Manhattan
2	3	01-01-2023	2	5	Lower Manhattan

```
1 print("\nDataset Info")
2 df.info()
```

```
Dataset Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 149116 entries, 0 to 149115
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   transaction_id         149116 non-null  int64
1   transaction_date       149116 non-null  object
2   transaction_qty        149116 non-null  int64
3   store_id               149116 non-null  int64
4   store_location         149116 non-null  object
5   product_id             149116 non-null  int64
6   unit_price             149116 non-null  float64
7   product_category       149116 non-null  object
8   product_type           149116 non-null  object
9   product_detail         149116 non-null  object
dtypes: float64(1), int64(4), object(5)
memory usage: 11.4+ MB
```

```
1 df.describe()
```

	transaction_id	transaction_qty	store_id	product_id	unit_pri
count	149116.000000	149116.000000	149116.000000	149116.000000	149116.000000
mean	74737.371872	1.438276	5.342063	47.918607	3.3822
std	43153.600016	0.542509	2.074241	17.930020	2.6587
min	1.000000	1.000000	3.000000	1.000000	0.8000
25%	37335.750000	1.000000	3.000000	33.000000	2.5000
50%	74727.500000	1.000000	5.000000	47.000000	3.0000
75%	112094.250000	2.000000	8.000000	60.000000	3.7500
max	149456.000000	8.000000	8.000000	87.000000	45.0000

```
1 print("\nDataset Shape")
2 print(df.shape)
```

Dataset Shape
(149116, 11)

```
1 print("\nColumn Names")
2 print(df.columns)
```

Column Names
Index(['transaction_id', 'transaction_date', 'transaction_qty', 'store_id',
 'store_location', 'product_id', 'unit_price', 'product_category',
 'product_type', 'product_detail'],
 dtype='object')

```
1 print("\nMissing Values")
2 print(df.isnull().sum())
```

Missing Values
transaction_id 0
transaction_date 0
transaction_qty 0
store_id 0
store_location 0
product_id 0
unit_price 0
product_category 0
product_type 0
product_detail 0
Month 0
dtype: int64

```
1 print("\nDuplicate Records")
2 print(df.duplicated().sum())
```

```
Duplicate Records
0
```

```
1 df.isnull().sum()
```

```

              0
transaction_id  0
transaction_date  0
transaction_qty  0
      store_id  0
      store_location  0
      product_id  0
      unit_price  0
product_category  0
      product_type  0
      product_detail  0
              Month  0

```

```
dtype: int64
```

✓ *Initial EDA*

Dataset Overview

The Coffee Shop Sales dataset contains transaction-level sales records collected from multiple store locations. The dataset includes information about transaction dates, transaction times, products sold, quantity purchased, unit prices, and store locations.

What was analyzed?

Column information Data types Statistical summary Missing values Duplicate records

Purpose of Initial EDA

The purpose of Initial EDA is to understand the structure and quality of the dataset before performing data cleaning and advanced analysis.

Key Observations

Total number of rows and columns were identified. Data types were verified. Missing values were checked. Duplicate records were detected. Statistical summary was generated for numerical columns.

Business Importance

Understanding the dataset structure helps ensure data quality and supports accurate analysis, visualization, and business decision-making.

Stage 2 – Data Cleaning and Pre-processing

✓ ***Tasks Performed***

- ✓ Handle Missing Values
- ✓ Check Duplicate Records
- ✓ Convert Date Columns
- ✓ Create Sales Feature
- ✓ Create Month Feature
- ✓ Create Quarter Feature
- ✓ Create Day Name Feature
- ✓ Create Hour Feature

```
1 df['transaction_date'] = pd.to_datetime(  
2     df['transaction_date'],  
3     dayfirst=True  
4 )
```

```
1 df[['transaction_date']].head()
```


	transaction_date
0	2023-01-01
1	2023-01-01
2	2023-01-01
3	2023-01-01
4	2023-01-01

```

1 df['Sales'] = df['transaction_qty'] * df['unit_price']
2
3 df['Month'] = df['transaction_date'].dt.month_name()
4
5 df['Quarter'] = df['transaction_date'].dt.quarter
6
7 df['Day_Name'] = df['transaction_date'].dt.day_name()

```

```

1 sales_q1 = df['Sales'].quantile(0.25)
2
3 sales_q2 = df['Sales'].quantile(0.50)
4
5 sales_q3 = df['Sales'].quantile(0.75)
6
7 df['Sales_Performance'] = pd.cut(
8     df['Sales'],
9     bins=[0,sales_q1,sales_q2,sales_q3,float('inf')],
10    labels=['Low','Medium','High','Excellent']
11 )
12
13 df['Sales_Performance'].value_counts()

```

	count
Sales_Performance	
Low	46384
High	45317
Medium	30885
Excellent	26530

dtype: int64

```
1 df[['Sales', 'Month', 'Quarter', 'Day_Name']].head()
```

	Sales	Month	Quarter	Day_Name
0	6.0	January	1	Sunday
1	6.2	January	1	Sunday
2	9.0	January	1	Sunday
3	2.0	January	1	Sunday
4	6.2	January	1	Sunday

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 149116 entries, 0 to 149115
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   transaction_id         149116 non-null int64
1   transaction_date       149116 non-null datetime64[ns]
2   transaction_qty        149116 non-null int64
3   store_id               149116 non-null int64
4   store_location         149116 non-null object
5   product_id             149116 non-null int64
6   unit_price             149116 non-null float64
7   product_category       149116 non-null object
8   product_type           149116 non-null object
9   product_detail         149116 non-null object
10  Month                   149116 non-null object
11  Sales                   149116 non-null float64
12  Quarter                 149116 non-null int32
13  Day_Name                149116 non-null object
14  Sales_Performance       149116 non-null category
dtypes: category(1), datetime64[ns](1), float64(2), int32(1), int64(4), object(6)
memory usage: 15.5+ MB
```

```
1 df.to_csv('cleaned_coffee_shop_sales.csv', index=False)
2
3 print("Data Cleaning Completed Successfully")
4 print("Final Shape:", df.shape)
```

```
Data Cleaning Completed Successfully
Final Shape: (149116, 15)
```

```
1 num_cols = ['transaction_qty', 'unit_price', 'Sales']
```

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
```

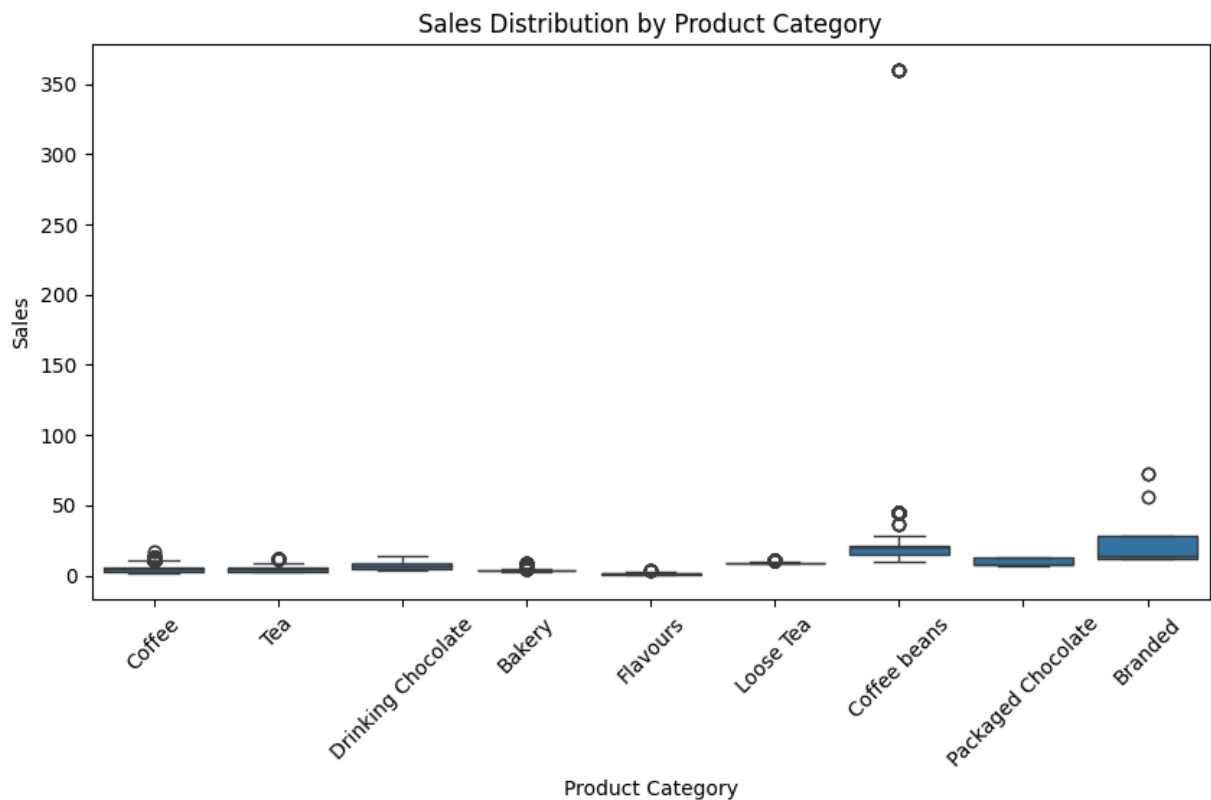
✓ *Summary of Preprocessing Performed*

- ✓ Converted transaction_date into datetime format
- ✓ Created Sales feature
- ✓ Created Month feature
- ✓ Created Quarter feature
- ✓ Created Day_Name feature
- ✓ Dataset prepared for Exploratory Data Analysis

Data Cleaning Completed Successfully

✓ **Stage 3 – Exploratory Data Analysis and Visualizations**

```
1 plt.figure(figsize=(10,5))
2
3 sns.boxplot(
4     x='product_category',
5     y='Sales',
6     data=df
7 )
8
9 plt.title('Sales Distribution by Product Category')
10 plt.xlabel('Product Category')
11 plt.ylabel('Sales')
12
13 plt.xticks(rotation=45)
14
15 plt.show()
```



✓ *Insights*

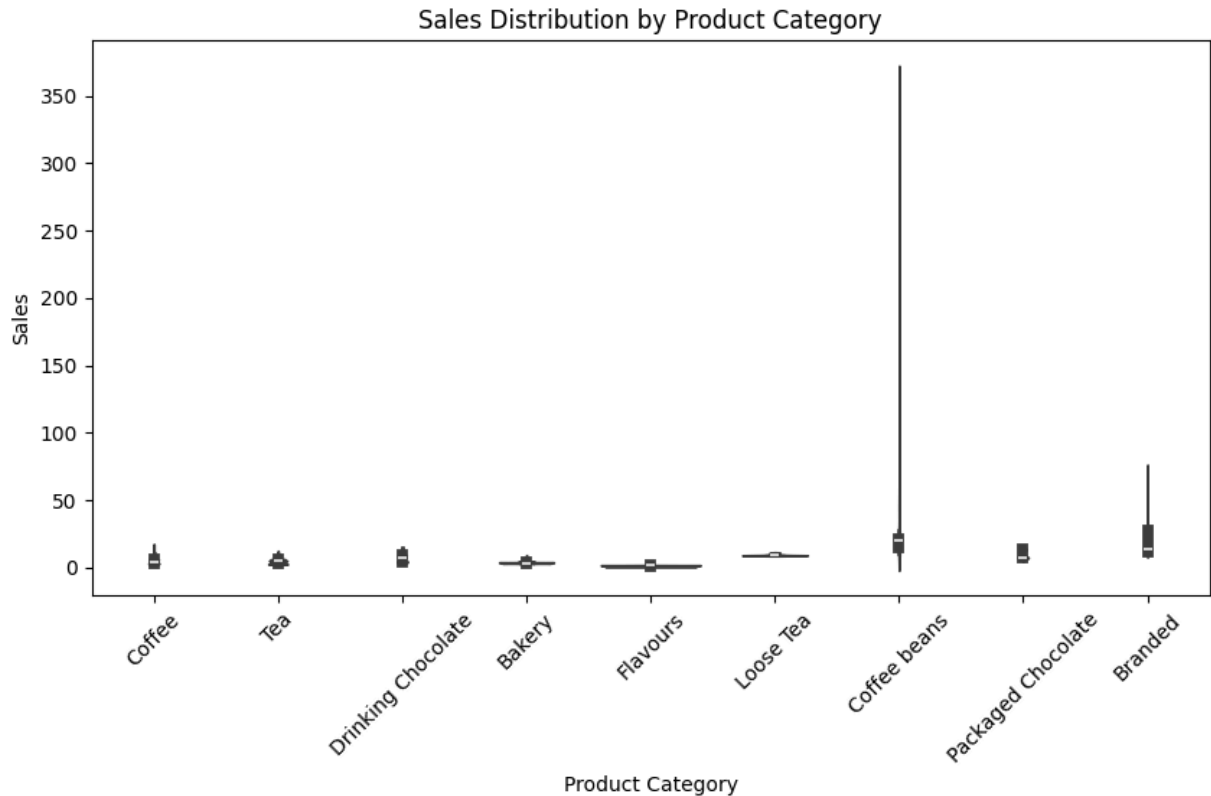
1. Sales distribution varies across different product categories.
2. Some product categories exhibit higher sales variability.
3. High-value sales transactions are observed in selected categories.
4. Customer purchasing patterns differ by product category.

```

1 plt.figure(figsize=(10,5))
2
3 sns.violinplot(
4     x='product_category',
5     y='Sales',
6     data=df
7 )
8

```

```
9 plt.title('Sales Distribution by Product Category')
10 plt.xlabel('Product Category')
11 plt.ylabel('Sales')
12
13 plt.xticks(rotation=45)
14
15 plt.show()
```



✓ *Insights*

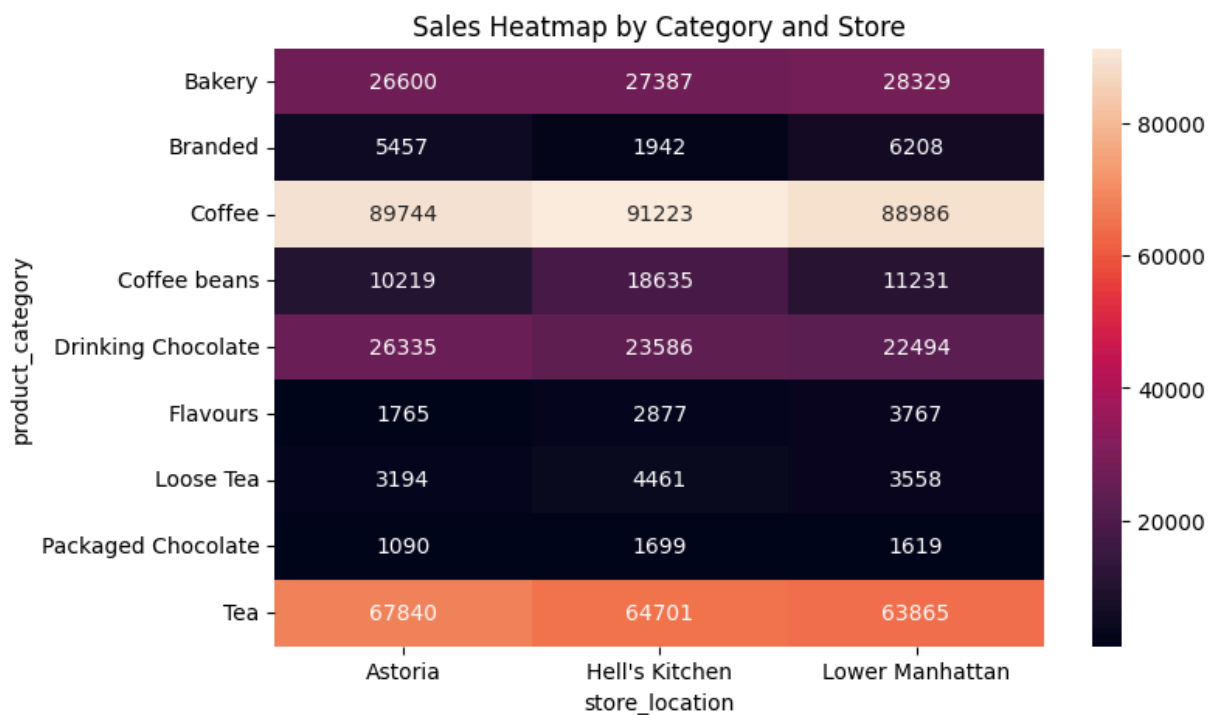
- 1.The majority of transactions are concentrated in low and medium sales ranges.
- 2.Coffee and Tea categories exhibit wider distributions, indicating higher customer demand.
- 3.Product categories show varying levels of sales density and transaction frequency.
- 4.Certain categories display occasional high-value transactions.

5.Customer purchasing behavior differs significantly across product categories.

```

1 pivot = pd.pivot_table(
2     df,
3     values='Sales',
4     index='product_category',
5     columns='store_location',
6     aggfunc='sum'
7 )
8
9 plt.figure(figsize=(8,5))
10
11 sns.heatmap(
12     pivot,
13     annot=True,
14     fmt='.0f'
15 )
16
17 plt.title('Sales Heatmap by Category and Store')
18
19 plt.show()

```



Insights

- 1.Revenue generation varies considerably among store locations.
- 2.Coffee products consistently generate strong revenue across all stores.
- 3.Hell's Kitchen store demonstrates superior sales performance.
- 4.Product category demand differs based on store location.
- 5.Store-specific strategies can improve business performance.

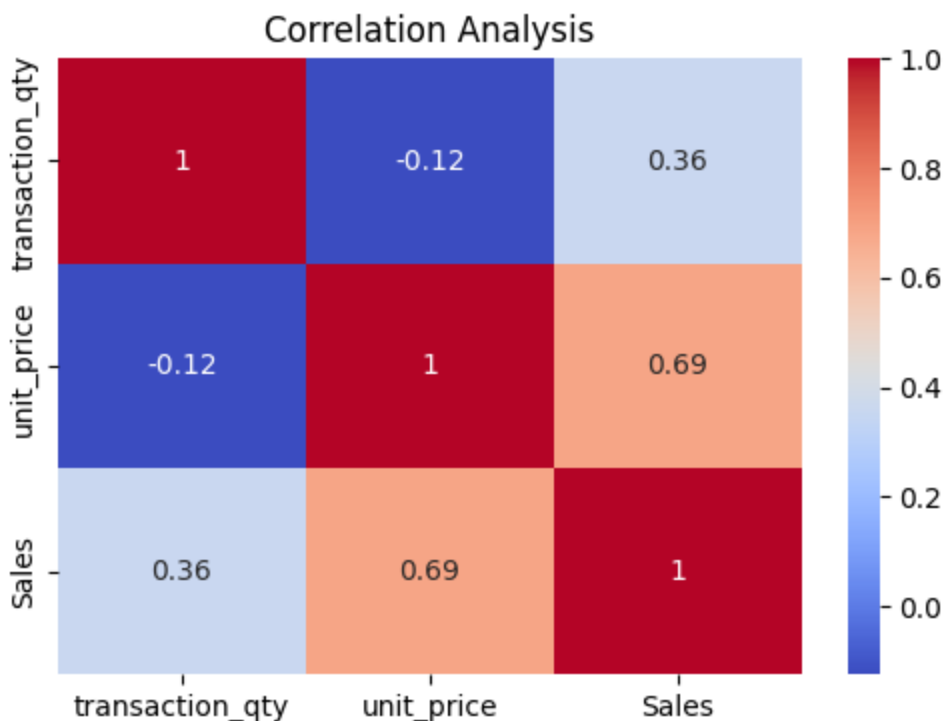
```
1 plt.figure(figsize=(8,5))
2
3 sns.scatterplot(
4     x='unit_price',
5     y='Sales',
6     data=df
7 )
8
9 plt.title('Unit Price vs Sales')
10 plt.xlabel('Unit Price')
11 plt.ylabel('Sales')
12
13 plt.show()
```



✓ *Insights*

- 1.A positive relationship exists between unit price and sales revenue.
- 2.Higher-priced products contribute significantly to overall revenue.
- 3.Customer purchasing behavior varies across price segments.
- 4.Premium products generate higher revenue despite lower purchase frequency.
- 5.Pricing strategy has a major impact on business profitability.


```
1 plt.figure(figsize=(6,4))
2
3 sns.heatmap(
4     df[['transaction_qty', 'unit_price', 'Sales']].corr(),
5     annot=True,
6     cmap='coolwarm'
7 )
8
9 plt.title('Correlation Analysis')
10
11 plt.show()
```

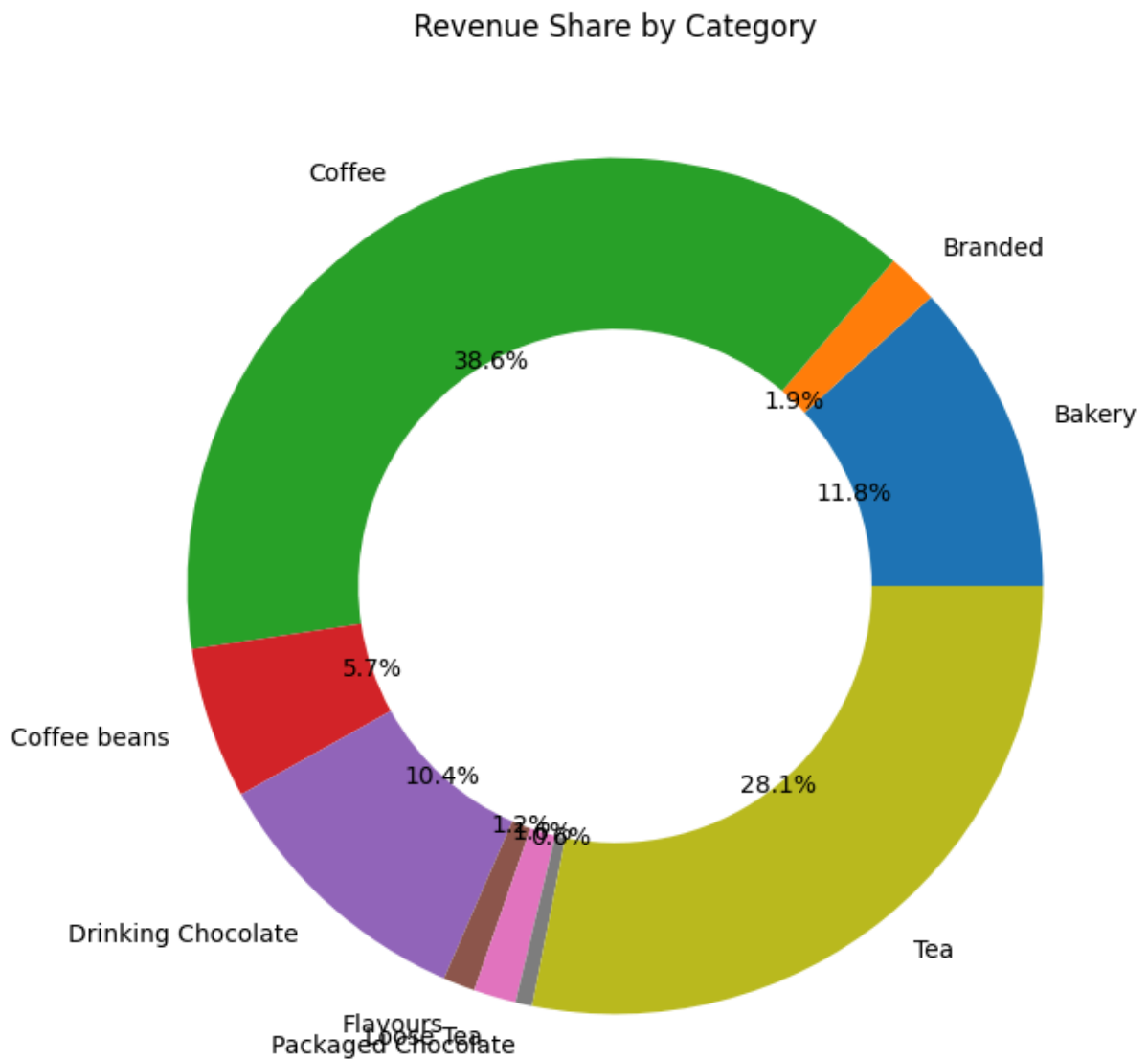


✓ *Insights*

- 1.Sales have a strong positive correlation with transaction quantity.
- 2.Unit price also contributes positively to revenue generation.
- 3.Transaction quantity is the most influential factor affecting sales.
- 4.Multiple variables collectively determine business performance.
- 5.Correlation analysis supports data-driven decision-making.

```
1 category_sales = df.groupby('product_category')['Sales'].sum()
2
3 plt.figure(figsize=(8,8))
4
```

```
5 plt.pie(  
6     category_sales.values,  
7     labels=category_sales.index,  
8     autopct='%1.1f%%',  
9     wedgeprops={'width':0.4}  
10 )  
11  
12 plt.title('Revenue Share by Category')  
13  
14 plt.show()
```



✓ *Insights*

Revenue Share by Category

- 1.Coffee contributes the highest percentage of total revenue (47.4%).
- 2.Tea is the second-largest revenue contributor.

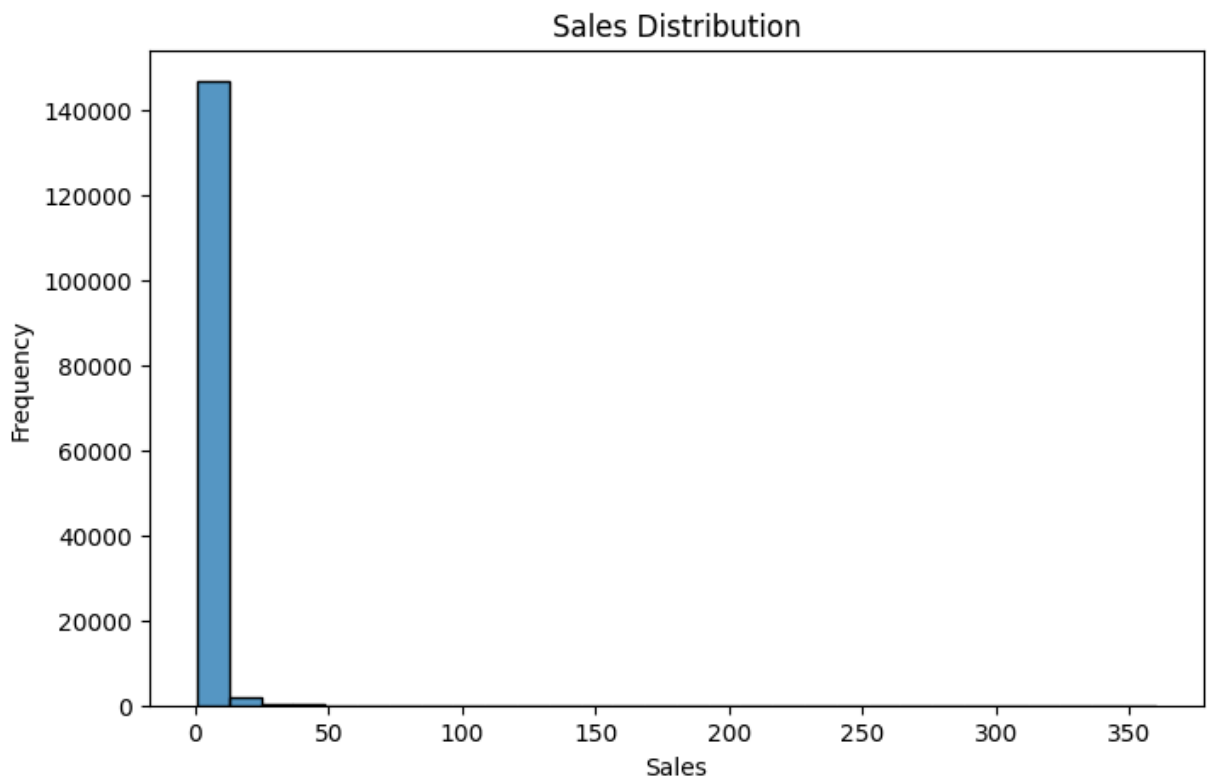
Top Products

- 1.Barista Espresso is the highest revenue-generating product.
- 2.Brewed Chai Tea and Gourmet Brewed Coffee contribute significantly to revenue.

Store Performance

- 1.Hell's Kitchen store records the highest average transaction value.
- 2.Lower Manhattan demonstrates moderate performance.
- 3.Astoria records comparatively lower average sales.

```
1 plt.figure(figsize=(8,5))
2
3 sns.histplot(
4     df['Sales'],
5     bins=30
6 )
7
8 plt.title('Sales Distribution')
9 plt.xlabel('Sales')
10 plt.ylabel('Frequency')
11
12 plt.show()
```

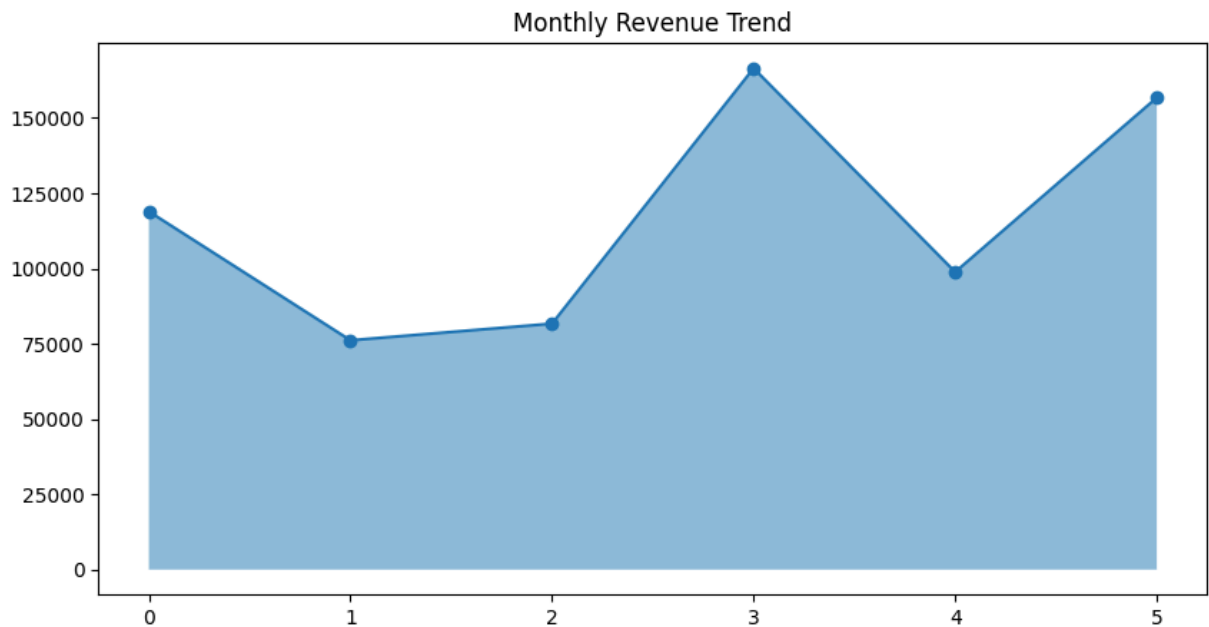


✓ *Insights*

1. Most transactions occur within low and medium sales ranges.
2. High-value transactions occur less frequently.
3. Customer spending patterns exhibit positive skewness.
4. The sales distribution is not uniform across customers.
5. Customer purchase behavior varies significantly.

```
1 monthly_sales = df.groupby('Month')['Sales'].sum()
2
3 plt.figure(figsize=(10,5))
4
5 plt.fill_between(
6     range(len(monthly_sales)),
7     monthly_sales.values,
8     alpha=0.5
9 )
10
11 plt.plot(monthly_sales.values, marker='o')
```

```
12  
13 plt.title('Monthly Revenue Trend')  
14  
15 plt.show()
```



✓ *Insights*

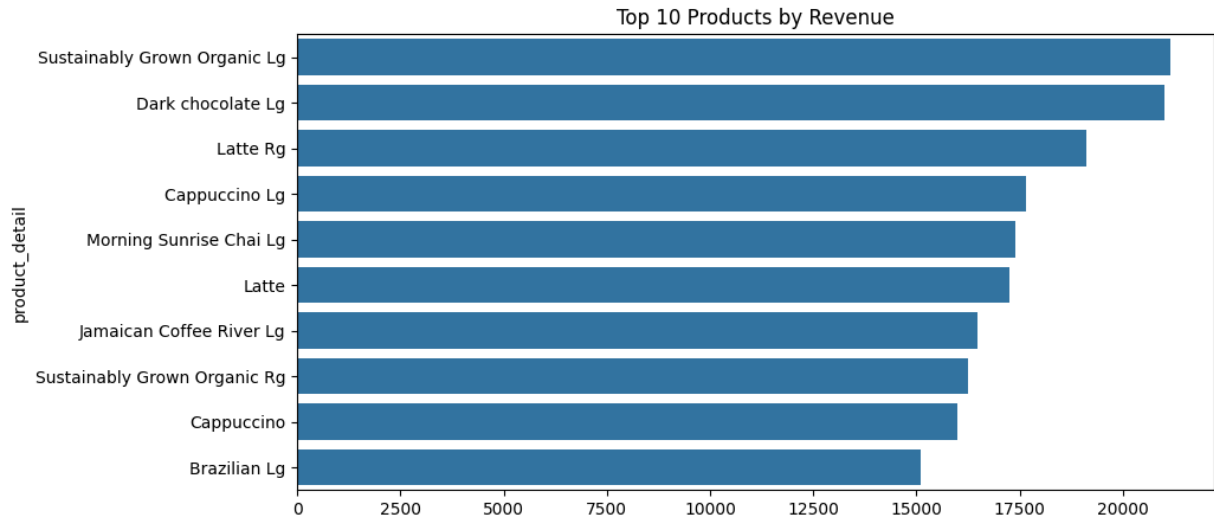
- 1.Monthly revenue fluctuates over time.
- 2.Seasonal patterns influence customer purchasing behavior.
- 3.Certain months generate significantly higher revenue.
- 4.Revenue trends help identify peak and low business periods.
- 5.Seasonal analysis supports inventory and marketing planning.

```
1 top_products = df.groupby('product_detail')['Sales'].sum()\n2                     .sort_values(ascending=False)\n3                     .head(10)\n4\n5 plt.figure(figsize=(10,5))\n6\n7 sns.barplot(
```

```

8     x=top_products.values,
9     y=top_products.index
10 )
11
12 plt.title('Top 10 Products by Revenue')
13
14 plt.show()

```



✓ *Insights*

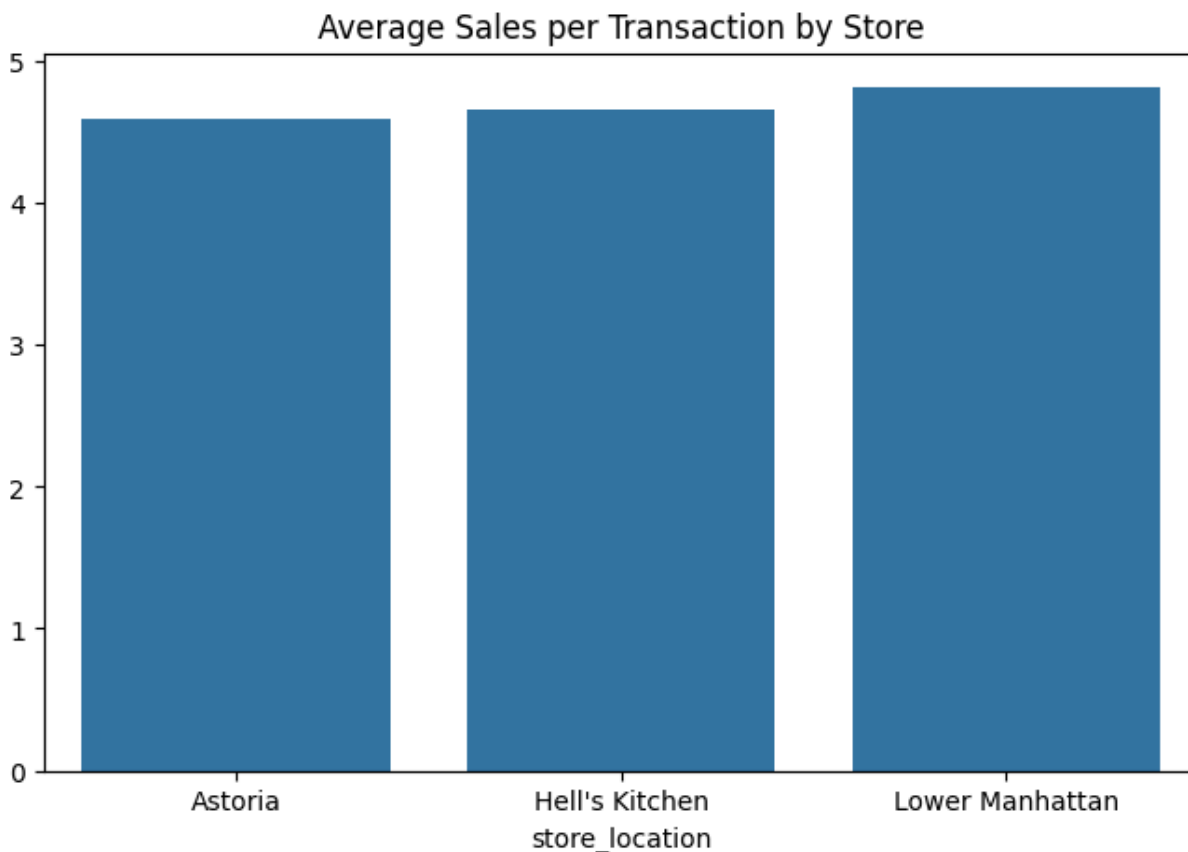
- 1.Barista Espresso is the highest revenue-generating product.
- 2.Brewed Chai Tea and Gourmet Brewed Coffee also contribute substantially.
- 3.A small number of products generate a significant portion of revenue.
- 4.Premium beverage products perform exceptionally well.
- 5.Product-focused marketing strategies can improve profitability

```

1 avg_sales = df.groupby('store_location')['Sales'].mean()
2
3 plt.figure(figsize=(8,5))
4

```

```
5 sns.barplot(  
6     x=avg_sales.index,  
7     y=avg_sales.values  
8 )  
9  
10 plt.title('Average Sales per Transaction by Store')  
11  
12 plt.show()
```



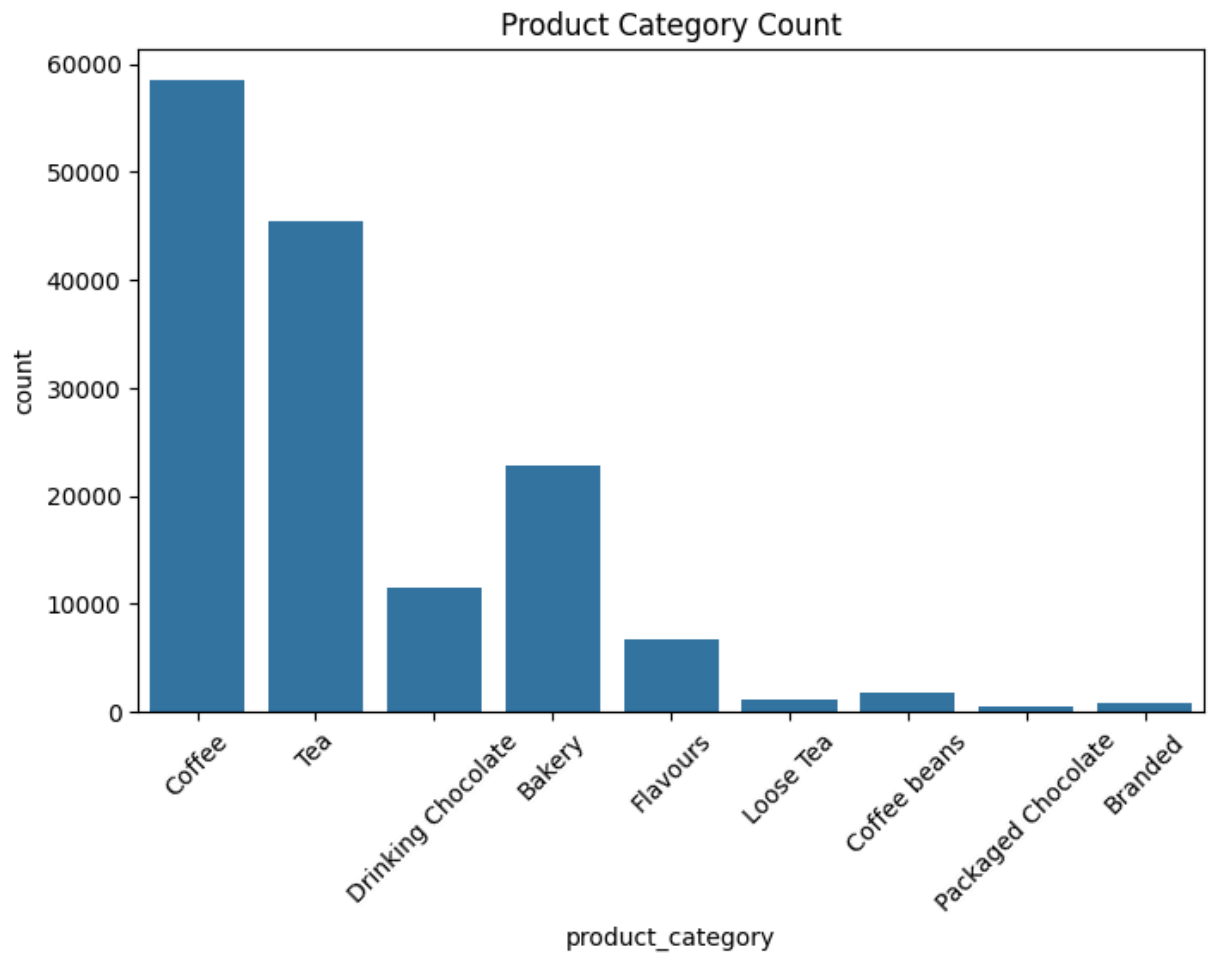
✓ *Insights*

- 1.Hell's Kitchen store records the highest average transaction value.
- 2.Lower Manhattan demonstrates moderate performance.
- 3.Astoria records comparatively lower average sales.
- 4.Customer spending behavior varies by location.
- 5.Store-level business strategies are required to maximize revenue.

```
1 plt.figure(figsize=(8,5))  
2 sns.countplot(  

```

```
3 x='product_category',  
4 data=df  
5 )  
6  
7 plt.title('Product Category Count')  
8 plt.xticks(rotation=45)  
9 plt.show()
```



✓ *Insights*

- 1.Coffee category contains the highest number of products available.
- 2.Flavours category contains the lowest number of products.
- 3.Product assortment varies significantly across categories.
- 4.Greater product variety may influence customer purchasing decisions.
- 5.Product portfolio management supports business growth.

✓ **STAGE 4 - Documentation, Insights & Presentation**



4.1 Skewness Analysis Before Treatment

```

1 num_cols = ['transaction_qty', 'unit_price', 'Sales']
2
3 print("Skewness Values")
4
5 for col in num_cols:
6     print(col, ":", df[col].skew())

```

```
Skewness Values
transaction_qty : 0.6799290776108677
unit_price : 0.08631946099821877
Sales : 0.9000585811656648
```

✓ Interpretation

The dataset exhibits positive skewness across all numerical variables.

Transaction quantity shows moderate positive skewness, while unit price and sales display high positive skewness due to a small number of high-value transactions.

These extreme observations may influence statistical analysis and visualization results.

4.2 Outlier Detection Before Treatment

```
1 for col in num_cols:
2
3     Q1 = df[col].quantile(0.25)
4     Q3 = df[col].quantile(0.75)
5
6     IQR = Q3 - Q1
7
8     lower = Q1 - 1.5 * IQR
9     upper = Q3 + 1.5 * IQR
10
11     outliers = df[(df[col] < lower) | (df[col] > upper)]
12
13     print(f"{col}: {len(outliers)} outliers")
```

```
transaction_qty: 0 outliers
unit_price: 0 outliers
Sales: 0 outliers
```

✓ Interpretation

Outlier analysis using the IQR method did not detect any significant outliers in the numerical variables. Therefore, no major treatment was required.

4.3 Outlier Treatment Using IQR Capping

```
1 for col in num_cols:
2
3     Q1 = df[col].quantile(0.25)
```

```
4     Q3 = df[col].quantile(0.75)
5
6     IQR = Q3 - Q1
7
8     lower = Q1 - 1.5 * IQR
9     upper = Q3 + 1.5 * IQR
10
11     df[col] = df[col].clip(lower, upper)
12
13 print("Outlier Treatment Completed")
```

Outlier Treatment Completed